2.1.12 Pure Functions

When you do an operation like `Map[f, expr]`, you have to specify a function *f* to apply. If there is already a function that does exactly what you want, then you can just give its name. Often, however, you may end up defining functions simply in order to use their names in `Map`.

This defines a function `h`.

```
In[1]:= h[x_] := f[x] + g[x]
```

Now you can use `h` in `Map`.

```
In[2]:= Map[h, {a, b, c}]
Out[2]= {f[a] + g[a], f[b] + g[b], f[c] + g[c]}
```

This section discusses a way to avoid defining functions like `h` simply in order to use their names in `Map`.

The function `h` in the example takes an argument *x* and returns the result `f[x] + g[x]`. If we are only going to use `h` in `Map`, its actual name is quite irrelevant. We could avoid explicitly defining `h` at all if we had some object that would simply compute `f[x] + g[x]` when applied to any expression. We could then give this object, rather than `h`, to `Map`.

The object `Function[x, f[x] + g[x]]` plays the role of a "pure function" in *Mathematica*. This object can be applied to any argument *a*, to give the result `f[a] + g[a]`.

This computes `f[x] + g[x]` for each element in the list.

```
In[3]:= Map[ Function[x, f[x] + g[x]], {a, b, c} ]
Out[3]= {f[a] + g[a], f[b] + g[b], f[c] + g[c]}
```

You can use `Function` to specify any kind of operation. For example, `Function[z, z^2]` corresponds to the operation of squaring an expression.

This applies the pure function `Function[z, z^2]` to the argument `1 + a`.

```
In[4]:= Function[z, z^2] [1 + a]
                      2
Out[4]= (1 + a)
```

You can treat a `Function` just like any other kind of expression.

```
In[5]:= t = Function[z, z^2]
                        2
Out[5]= Function[z, z ]
```

Now `t` stands for the pure function `Function[z, z^2]`, so this gives the square of the argument `1 + a`.

```
In[6]:= t[1 + a]
                      2
Out[6]= (1 + a)
```

You can specify any operation as a "pure function" in the form `Function[`*variable*`,` *operation*`]`. Whenever you provide an argument for the pure function, it substitutes the argument for *variable*, and performs the *operation*. (If you know LISP or formal logic, you will recognize `Function` as being analogous to a λ expression. It is also close to the mathematical notion of an "operator".)

You can specify any operation as a pure function.

```
In[7]:= Function[e, Expand[e^2]] [ 1 + x ]

Out[7]= 1 + 2 x + x^2
```

You can use `Function` whenever you need to give a function.

```
In[8]:= Select[ {1, 7, 8, 2, 9}, Function[n, n > 4] ]
Out[8]= {7, 8, 9}
```

`Function[`*variable*`,` *operation*`]`	a pure function that performs the operation on any argument you provide
`Function[{`x_1`,` x_2`, ...}, `*operation*`]`	a pure function that can take several arguments

A way to represent functions without giving them names.

The pure functions we have discussed so far take just one argument. You can also set up pure functions that can take several arguments.

This pure function expects two arguments.

```
In[9]:= t = Function[{x, y}, x + 2 y]
Out[9]= Function[{x, y}, x + 2 y]
```

`a` is substituted for `x`, and `b` for `y`, in the pure function.

```
In[10]:= t[a, b]
Out[10]= a + 2 b
```

If you give too many arguments, the remaining ones are ignored.

```
In[11]:= t[a, b, c]
Out[11]= a + 2 b
```

If you give too few arguments, you get a message.

```
In[12]:= t[a]
Function::count:
      Too many parameters {x, y} to be filled from
       Function[{x, y}, x + 2 y][a].
Out[12]= a + 2 y
```